



CS50
Week 3
Algorithms



Outlines :

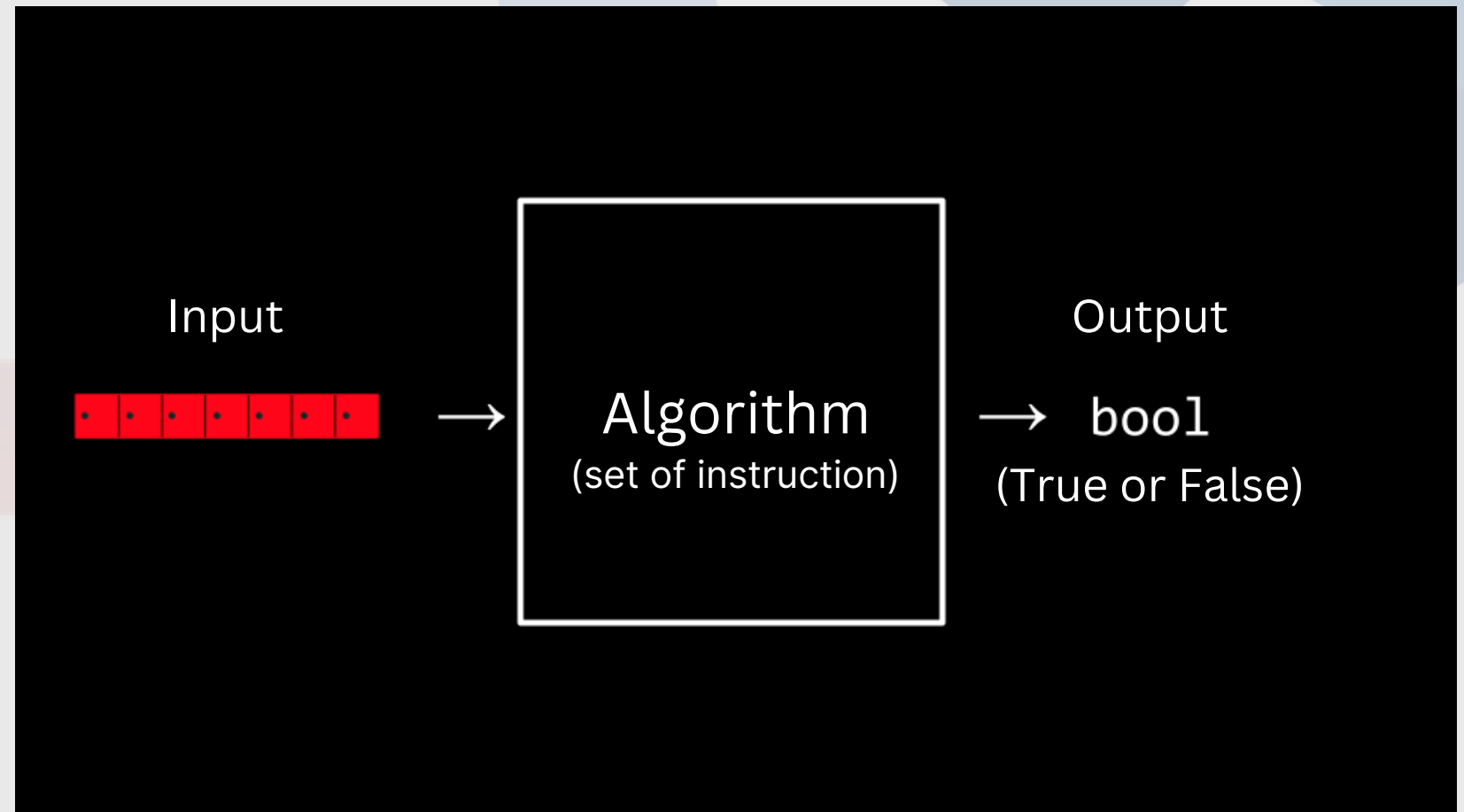
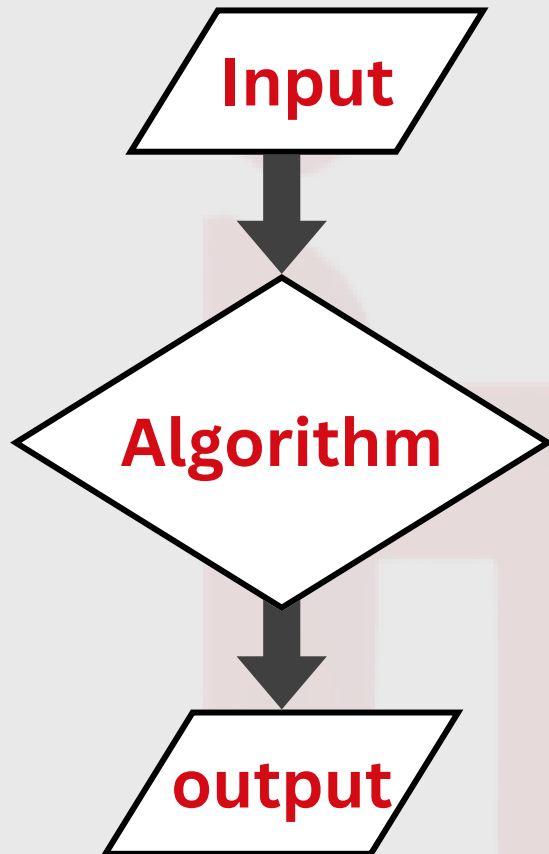
- **Algorithms**
- **Linear Search**
- **Binary Search**
- **Time & Space Complexity**

What is Algorithm



- **Algorithm :**

An algorithm is a step-by-step procedure for solving a problem or performing a task, with a clear set of instructions that produce a specific output from given inputs.



Outlines :

- Algorithms
- **Linear Search**
- Binary Search
- Time & Space Complexity

• Linear Search :

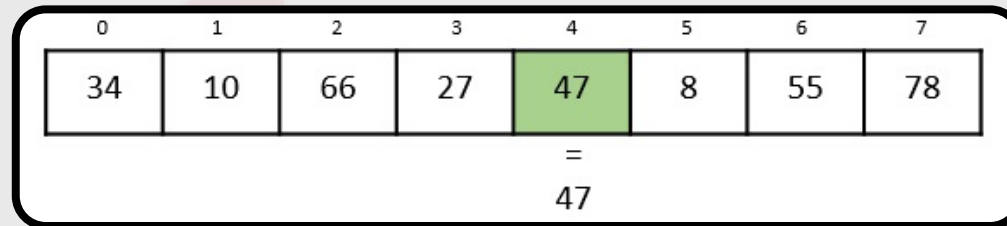
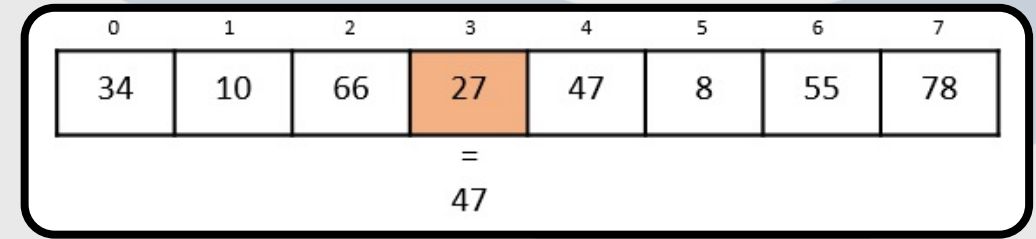
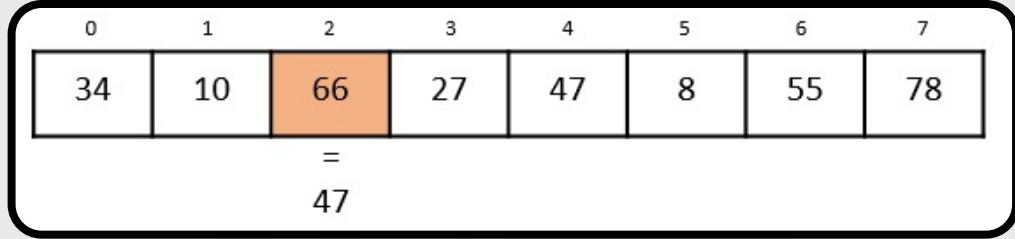
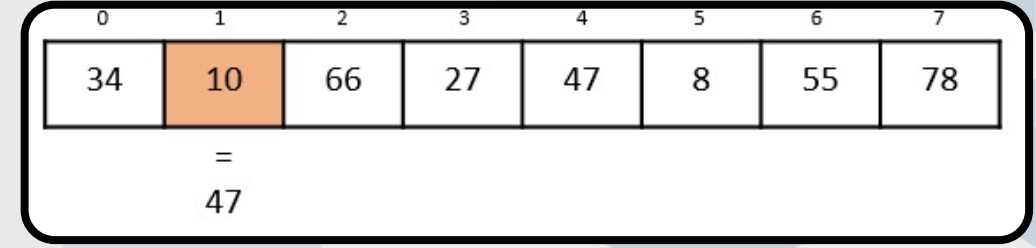
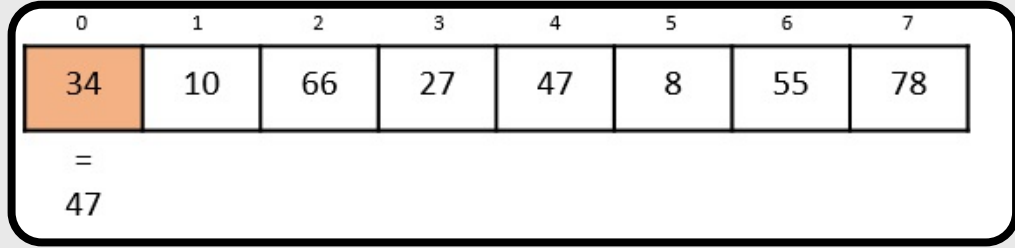
Linear search is a simple algorithm that checks each element in a list sequentially until the desired element is found or the list ends.

For example, given an array, if we want to search for the element 47 :

- We loop through each element and check if it is found.
- If we reach the end of the array and have not found the element, then it does not exist.

0	1	2	3	4	5	6	7
34	10	66	27	47	8	55	78

Steps :





Pseudocode :

```
For each door from left to right
  If 50 is behind door
    Return true
Return false
```

To make it look like the code :

```
For i from 0 to n-1
  If 50 is behind doors[i]
    Return true
Return false
```

[Visit this for visualization.](#)



Linear Search

Code :

(Example#1)

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  int main()
5  {
6      int number = 10;
7      int arr[5] = {1, 7, 7, 15, 10};
8      bool isNumberFound = false;
9      for (int i = 0; i < 5; i++)
10     {
11         if (arr[i] == number)
12         {
13             isNumberFound = true;
14             break;
15         }
16     }
17     if (isNumberFound)
18     {
19         cout << "Number found in the array" << endl;
20     }
21     else
22     {
23         cout << "Number not found in the array" << endl;
24     }
25     return 0;
26 }
27
```



Example#2 :

Given an array of integers, count the number of even numbers.

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 int countEvenNumbers(int arr[], int size)
6 {
7     int count = 0;
8     for (int i = 0; i < size; i++)
9     {
10         if (arr[i] % 2 == 0)
11         {
12             count++;
13         }
14     }
15     return count;
16 }
17 int main()
18 {
19     int size = 5;
20     int arr[5] = {1, 7, 7, 15, 10};
21     cout << countEvenNumbers(arr, size) << endl;
22     return 0;
23 }
24
```



Example#3 :

Given an array of integers, find the minimum value.

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  int findMinimum(int arr[], int size)
6  {
7      int min = arr[0];
8      for (int i = 1; i < size; i++)
9      {
10         if (arr[i] < min)
11         {
12             min = arr[i];
13         }
14     }
15     return min;
16 }
17 int main()
18 {
19     int size = 5;
20     int arr[5] = {1, 7, 7, 15, 10};
21     cout << findMinimum(arr, size) << endl;
22     return 0;
23 }
```

Outlines :

- Algorithms
- Linear Search
- **Binary Search**
- Time & Space Complexity

• Binary Search :

Binary search is an efficient algorithm for finding an item from a sorted array by repeatedly dividing the search interval in half, knowing that the left half contains smaller elements and the right half contains larger elements.

Binary Search **Example :**

0					10						
10	30	40	50	70	80	90	100	140	180	200	

0					10						
10	30	40	50	70	80	90	100	140	180	200	

0			4	
10	30	40	50	70

0			4	
10	30	40	50	70

3	4
50	70



Pseudocode :

```
If no doors left
    Return false
If 50 is behind middle door
    Return true
Else if 50 < middle door
    Search left half
Else if 50 > middle door
    Search right half
```

To make it look likes the code :

```
If no doors left
    Return false
If 50 is behind doors[middle]
    Return true
Else if 50 < doors[middle]
    Search doors[0] through doors[middle - 1]
Else if 50 > doors[middle]
    Search doors[middle + 1] through doors[n - 1]
```

[Visit this for visualization.](#)



Binary Search

Code :

(Example#1)

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  int main()
6  {
7      int size = 5, target = 7;
8      int arr[5] = {1, 7, 7, 15, 10};
9      bool flag = false;
10     int left = 0, right = size - 1;
11     while (left <= right){
12         int mid = (left + right) / 2;
13         if (arr[mid] == target){
14             flag = true;
15             break;
16         }
17         else if (arr[mid] < target){
18             left = mid + 1;
19         }
20         else{
21             right = mid - 1;
22         }
23     }
24     if (flag){
25         cout << "Number found" << endl;
26     }
27     else{
28         cout << "Number not found" << endl;
29     }
30     return 0;
31 }
```



Example#2 :

Given a sorted array of integers, find the index of the first occurrence of a given integer x, -1 if not found.

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 int findFirstOccurrence(int arr[], int size, int target){
6     int low = 0, high = size - 1;
7     int result = -1;
8
9     while (low <= high){
10         int mid = low + (high - low) / 2;
11
12         if (arr[mid] == target){
13             result = mid;
14             high = mid - 1;
15         }
16         else if (arr[mid] > target){
17             high = mid - 1;
18         }
19         else{
20             low = mid + 1;
21         }
22     }
23
24     return result;
25 }
26 int main()
27 {
28     int size = 5, target = 7;
29     int arr[5] = {1, 7, 7, 15, 10};
30     cout << findFirstOccurrence(arr, size, target) << endl;
31     return 0;
32 }
```




Example#3 :

Given a sorted array of integers, find the smallest element greater than a given integer x.

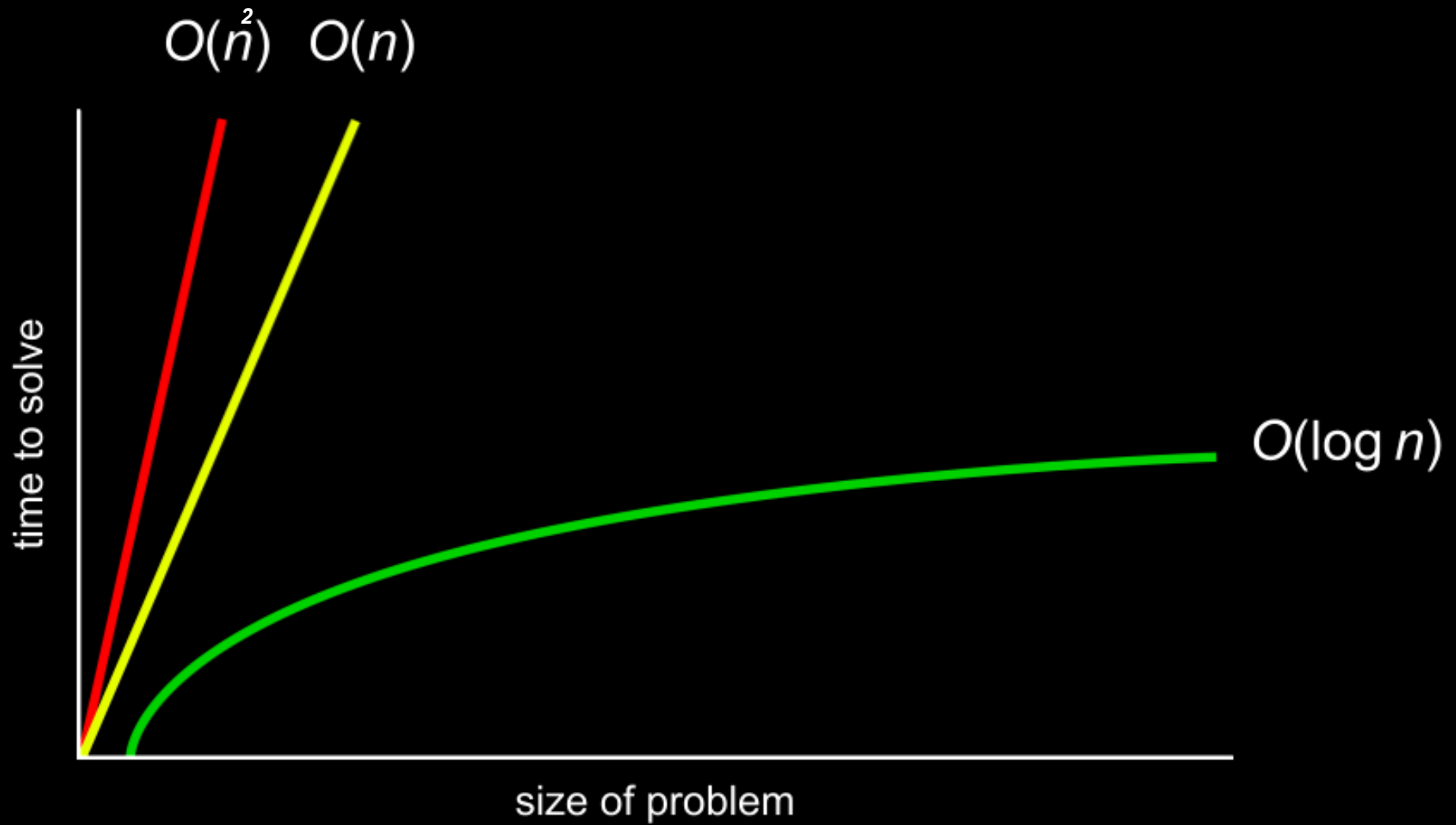
```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  int findSmallestElementGreaterThan(int arr[], int size, int target){
6      int left = 0, right = size - 1;
7      int result = -1;
8      while (left <= right){
9          int mid = left + (right - left) / 2;
10         if (arr[mid] > target){
11             result = mid;
12             right = mid - 1;
13         }
14         else{
15             left = mid + 1;
16         }
17     }
18     return result;
19 }
20 int main()
21 {
22     int size = 5, target = 7;
23     int arr[5] = {1, 7, 7, 15, 10};
24     cout << findSmallestElementGreaterThan(arr, size, target) << endl;
25     return 0;
26 }
```

Outlines :

- Algorithms
- Linear Search
- Binary Search
- **Time & Space Complexity**



Running Time :





Time Complexity :

Key Differences	Linear Search	Binary Search
Time Complexity	$O(n)$ (Linear time)	$O(\log n)$ (Logarithmic time)
Suitability for Sorted Data	Works with both unsorted and sorted data.	Primarily suitable for sorted data.
Memory Usage	$O(1)$ (Constant memory)	$O(1)$ (Constant memory)
Search Speed	Slower for large datasets, may have to examine all elements in the worst case.	Faster, especially for large datasets, significantly reduces search time.



Space Complexity :



Best Case

Worst Case

TIME COMPLEXITY



SPACE COMPLEXITY

- Measures the amount of time an algorithm takes to complete as a function of the input size.
- Provides an estimation of how the running time of an algorithm increases as the input size grows.

- Refers to the amount of memory an algorithm uses to execute, again as a function of the input size.
- Estimates how the memory consumption changes with different input sizes.



**Support
Team**

